

---

# Page 1 - Spring AI Fundamentals

Core concepts interviewers expect you to explain clearly.

## Q1. What is Spring AI?

**Answer:** Spring AI is a Spring ecosystem project that helps Java developers build AI-powered applications. It provides abstractions for chat models, embeddings, vector stores, prompt templates, advisors, and RAG workflows so you can work with AI providers using familiar Spring Boot patterns.

## Q2. Why use Spring AI instead of calling an AI API directly?

**Answer:** Direct API calls work, but Spring AI gives you cleaner integration with dependency injection, configuration properties, reusable ChatClient beans, provider switching, prompt templates, and vector store support. It keeps AI code closer to normal Spring Boot architecture.

## Q3. What is ChatClient?

**Answer:** ChatClient is the fluent API used to send prompts to a chat model. You can add system messages, user messages, advisors, options, and then call the model to receive generated content.

## Q4. What is the difference between ChatClient and ChatModel?

**Answer:** ChatModel is the lower-level model abstraction that represents the actual AI provider. ChatClient is a higher-level, developer-friendly API built on top of ChatModel.

## Q5. How do OpenAI and Ollama differ in Spring AI?

**Answer:** OpenAI calls cloud-hosted models such as GPT-4o mini using an API key. Ollama runs local models such as Qwen, Llama, or DeepSeek on your machine. Spring AI can configure both, but you must choose the correct bean when multiple providers exist.

---

## Page 2 - PromptTemplate and ChatClient

Prompt construction is where many Spring AI bugs happen.

### Q1. What is PromptTemplate?

**Answer:** PromptTemplate is a reusable template containing placeholders such as {topic}, {context}, or {question}. You render it with a Map so the placeholders are replaced with real values before sending the prompt to the model.

### Q2. Why is PromptTemplate better than string concatenation?

**Answer:** It keeps large prompts readable, reusable, and easier to maintain. It also reduces messy code when a prompt has multiple variables such as context, question, tone, and formatting rules.

### Q3. What is the difference between system() and user()?

**Answer:** system() defines the assistant's behavior, rules, or role. user() contains the actual user request. For example, system can say 'Use only context,' while user asks 'What is ChatClient?'

### Q4. What does call().content() do?

**Answer:** call() sends the prompt to the configured model. content() extracts only the generated text from the response. If you need metadata or token usage, use the richer response methods instead of content().

### Q5. How do advisors help?

**Answer:** Advisors add behavior around chat calls. SimpleLoggerAdvisor, for example, logs the request and response, which is useful for debugging prompts, retrieved context, and unexpected model answers.

---

## Page 3 - Embeddings and Vector Stores

Embeddings and vector stores power semantic search and RAG.

### Q1. What is an embedding?

**Answer:** An embedding is a numerical vector representation of text. Similar meanings produce vectors that are close together, allowing the system to search by meaning instead of exact keywords.

### Q2. What is an EmbeddingModel?

**Answer:** EmbeddingModel is the Spring AI abstraction that converts text into embeddings. OpenAI can use models such as text-embedding-3-small, and Ollama can use local embedding models such as nomic-embed-text.

### Q3. What is a VectorStore?

**Answer:** A VectorStore stores documents and their embeddings. It supports similarity search, which retrieves documents whose vectors are closest to the user's query vector.

### Q4. What is metadata in a Document?

**Answer:** Metadata is extra structured information stored with the document, such as title, genre, year, source, or knowledgeBase. It helps filter search results and organize multiple knowledge bases.

### Q5. Why can multiple EmbeddingModel beans cause errors?

**Answer:** If OpenAI and Ollama embeddings are both configured, Spring sees more than one EmbeddingModel. You must use @Qualifier for a specific injection point or mark one EmbeddingModel as @Primary for default use.

---

## Page 4 - How RAG Works

RAG stands for Retrieval-Augmented Generation. It combines search with generation.

### Q1. What problem does RAG solve?

**Answer:** LLMs can hallucinate or lack private project knowledge. RAG gives the model relevant documents at request time, so the answer is grounded in your own data.

### Q2. Explain the RAG workflow.

**Answer:** The user asks a question. The application embeds that question. The vector store finds similar documents. The retrieved text is combined into context. A prompt template sends context plus the question to the LLM. The LLM answers using only that context.

### Q3. Why is the user's question included in the final prompt?

**Answer:** Similarity search uses the question to retrieve documents, but the LLM also needs to see the actual question. Without it, the model may just summarize the context instead of answering the user.

### Q4. Why use a similarity threshold?

**Answer:** topK always returns the closest documents, even if they are not truly relevant. A similarity threshold prevents weak matches from being used. If no documents pass the threshold, the app can return 'I don't know.'

### Q5. What is context injection?

**Answer:** Context injection is the step where retrieved document text is joined into one string and inserted into the prompt template under a Context section.

---

## Page 5 - PGVector and Spring Boot Design

PGVector lets PostgreSQL act as a vector database.

### Q1. What is PGVector?

**Answer:** PGVector is a PostgreSQL extension that stores vectors and supports similarity search. In Spring AI, it can be used as a VectorStore implementation for RAG.

### Q2. What dependencies are commonly needed for PGVector?

**Answer:** You usually need the Spring AI PGVector starter and the PostgreSQL JDBC driver. Without the PostgreSQL driver, Spring Boot may fail with 'Failed to determine a suitable driver class.'

### Q3. How should application.yml be structured?

**Answer:** Datasource settings must be under spring.datasource, while PGVector settings should be under spring.ai.vectorstore.pgvector. Incorrect indentation can prevent auto-configuration from working.

### Q4. How do you support multiple knowledge bases?

**Answer:** Store a metadata field such as knowledgeBase=ai or knowledgeBase=movies. Then use filterExpression, for example knowledgeBase == 'ai', so similarity search only searches the correct document group.

### Q5. What is a good service design?

**Answer:** Keep ChatClient, EmbeddingModel, and VectorStore injected through constructor injection. Use @Primary for default beans or @Qualifier when multiple providers exist.

---

## Page 6 - Intermediate Interview Scenarios

Scenario-style questions that test real implementation understanding.

### Q1. Your RAG app answers from unrelated documents. What do you check?

**Answer:** Check whether the final prompt includes the user's question, whether `similarityThreshold` is set, whether the metadata filter is correct, and whether the retrieved context actually contains the answer.

### Q2. Your `filterExpression` fails with 'no viable alternative at input ai.' Why?

**Answer:** The string value is missing quotes. Use `knowledgeBase == 'ai'`, not `knowledgeBase == ai`.

### Q3. The metadata column shows `{}`. Why?

**Answer:** You probably created documents with `new Document(text)` only. To store metadata, use `new Document(text, Map.of('title', '...', 'topic', '...'))`.

### Q4. What does this code do: `documents.stream().map(Document::getText).collect(Collectors.joining(' '))`?

**Answer:** It extracts the text from each retrieved `Document` and joins all document texts into one context string separated by blank lines. That context is then inserted into the RAG prompt.

### Q5. How would you answer an interviewer asking you to explain your Spring AI project?

**Answer:** I built a RAG workflow where documents are embedded and stored in `PGVector`. When a user asks a question, the app performs similarity search with metadata filtering, builds context from retrieved documents, renders a `PromptTemplate` with context and question, and sends it to `ChatClient` for a grounded response.